

Experiment No. 13

Title

Minimax Algorithm for Gaming Using Artificial Intelligence

Aim

To develop a Python program to implement the Minimax Algorithm for game-playing applications.

Objective

The objective of this experiment is to understand the Minimax Algorithm and implement it to select the optimal move by evaluating the possible outcomes of a game.

Algorithm

Step 1: Create the game tree.

Step 2: Assign utility values to the leaf nodes.

Step 3: Apply the Minimax algorithm recursively.

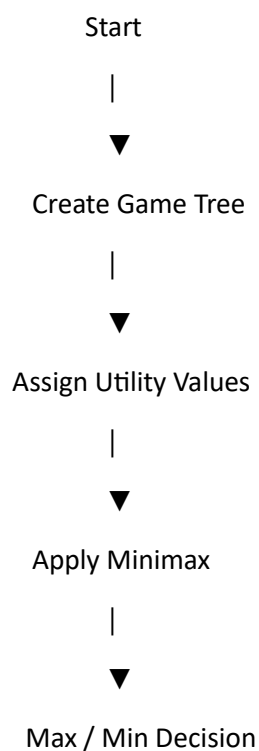
Step 4: Max player selects the maximum value.

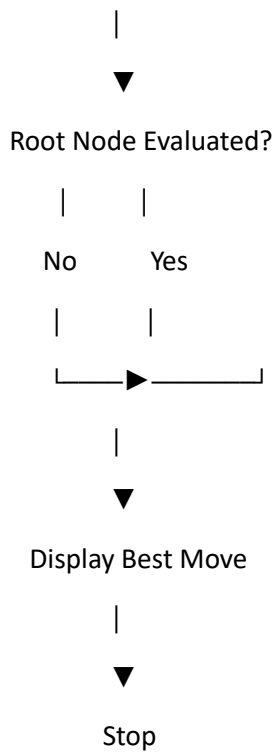
Step 5: Min player selects the minimum value.

Step 6: Continue until the root node is evaluated.

Step 7: Display the optimal move.

Flowchart





Python Program

```
def minimax(depth, node, maximizing, values):

    if depth == 3:
        return values[node]

    if maximizing:
        return max(
            minimax(depth + 1, node * 2, False, values),
            minimax(depth + 1, node * 2 + 1, False, values)
        )
    else:
        return min(
            minimax(depth + 1, node * 2, True, values),
            minimax(depth + 1, node * 2 + 1, True, values)
        )
```

```
values = [3, 5, 2, 9, 12, 5, 23, 23]
```

```
result = minimax(0, 0, True, values)
```

```
print("Optimal Value:", result)
```

Sample Output

Optimal Value: 12

Result

The optimal game value was successfully determined using the Minimax Algorithm.

Conclusion

Thus, the Minimax Algorithm was successfully implemented. It evaluates all possible game states and selects the optimal move by assuming that both players play optimally. This algorithm is widely used in game-playing AI applications such as Chess and Tic Tac Toe.